

Functional Programming using Lambda Expression

Lập trình chức năng sử dụng biểu thức Lambda

Nêu vấn đề

- Ví dụ: Danh sách nhân viên → Lọc nhân viên trên 18 tuổi → In tên
- **OOP trả lời: "Khách hàng là ai?"**
- **Functional Programming trả lời: "Xử lý khách hàng như thế nào?"**

OOP và FP

- OOP quản lý trạng thái và mô hình nghiệp vụ
- FP tối ưu cách xử lý dữ liệu
- Java hiện đại = OOP + FP

Tư duy OOP

- Class, Object
- Encapsulation
- Inheritance
- Polymorphism

Thế giới được mô hình hóa bằng đối tượng.

Tư duy Functional Programming

Input → Function → Output

Đặc trưng:

- **Pure Function (Cùng Input → Cùng Output)**
- **Immutable Data (Dữ liệu bất biến)**
- **Higher-order Function (Nhận hàm khác làm đối số)**
- **Lambda Expression (Cách viết ngắn gọn một hàm)**

Functional Programming

- Từ Java 8 hỗ trợ lập trình chức năng thông qua các biểu thức lambda.
- Trước Java 8, viết mã theo phong cách bắt buộc (Imperative Programming).
- Nhưng sau Java 8, phong cách thay đổi hướng tới lập trình chức năng

Phong cách lập trình

- Imperative Programming (Lập trình mệnh lệnh)
 - Chỉ rõ từng bước để máy tính thực hiện.
- Declarative Programming (Lập trình khai báo)
 - Mô tả kết quả mong muốn, còn hệ thống tự quyết định cách thực hiện.

Imperative Programming

```
List<Integer> numbers = Arrays.asList(1, 2, 3);
```

```
List<Integer> result = new ArrayList<>();
```

```
for (Integer n : numbers) {  
    if (n % 2 == 0) {  
        result.add(n * 2);  
    }  
}
```

```
System.out.println(result);
```

1. Tạo danh sách kết quả
2. Duyệt từng phần tử
3. Kiểm tra điều kiện
4. Nhân đôi giá trị
5. Thêm vào danh sách

=> Tập trung vào **quy trình thực hiện**.

Ưu điểm

- Dễ hiểu với người mới.
- Kiểm soát chi tiết cách chương trình chạy.
- Dễ tối ưu ở mức thấp.

Nhược điểm

- Mã dài.
- Dễ phát sinh lỗi do quản lý trạng thái.
- Khó bảo trì khi logic phức tạp.

Declarative Programming

```
List<Integer> result = numbers.stream()
```

```
.filter(n -> n % 2 == 0)
```

```
.map(n -> n * 2)
```

```
.toList();
```

Đọc như tiếng Anh:

Lấy danh sách số → lọc số chẵn → nhân đôi → chuyển thành danh sách.

Không quan tâm vòng lặp chạy thế nào.

=> Tập trung vào **kết quả mong muốn**.

Ưu điểm

- Code ngắn gọn.
- Dễ đọc.
- Ít lỗi liên quan đến trạng thái.
- Dễ song song hóa (parallel processing).

Nhược điểm

- Khó hiểu hơn lúc mới học.
- Debug đôi khi khó hơn.
- Có thể có overhead trong một số trường hợp.

So sánh phong cách lập trình

Imperative

Mô tả cách làm

Tập trung vào "How"

Dùng vòng lặp, biến trạng thái

Kiểm soát chi tiết

Code thường dài hơn

Declarative

Mô tả kết quả

Tập trung vào "What"

Dùng biểu thức, pipeline

Trừu tượng cao

Code thường ngắn gọn hơn

Trong thực tế Java

Lập trình Java thường là **kết hợp cả hai**:

- Xử lý nghiệp vụ đơn giản → **Declarative**
- Thuật toán phức tạp, cấu trúc dữ liệu, tối ưu hiệu năng → **Imperative**

Biểu Thức Lambda

- Biểu thức Lambda là một cách để biểu diễn các hàm ẩn danh (anonymous functions) trong Java
- Cho phép truyền một hàm dưới dạng tham số hoặc lưu trữ một hàm như một biến mà không cần khai báo một lớp cụ thể
- Cú pháp cơ bản của biểu thức Lambda trong Java như sau:

(parameter) -> expression

- Parameter: Danh sách các tham số của hàm.
 - ->: Toán tử Lambda, phân tách tham số và thân hàm.
 - Expression: Thân hàm, chứa logic xử lý.
- Ví dụ đơn giản: $(x, y) \rightarrow x + y$

Cú pháp chi tiết của Biểu thức Lambda

Biểu thức Lambda có thể được viết theo nhiều cách tùy thuộc vào tình huống:

- Không tham số:
`() -> System.out.println("Hello, Lambda!");`
- Một tham số (có thể bỏ qua dấu ngoặc): `x -> x * x`
- Nhiều tham số: `(x, y) -> x + y`
- Thân hàm phức tạp (dùng dấu ngoặc nhọn và từ khóa `return`):

```
(x, y) -> {  
    int sum = x + y;  
    return sum;  
}
```

Ví dụ

Imperative

Lambda Expression

```
public void printHello() {  
    System.out.println("Hello");  
}
```

```
() -> {  
    System.out.println("Hello");  
}
```

- Khi chỉ có một dòng biểu thức duy nhất bên trong hàm lambda, chúng ta có thể sử dụng cú pháp lambda biểu thức thay vì cú pháp câu lệnh lambda để đơn giản hơn.

```
public void printHello() {  
    System.out.println("Hello");  
}
```

```
() -> System.out.println("Hello");
```

Ví dụ 2

Imperative

```
public void add(int num1, int num2) {  
    int result = num1 + num2;  
    System.out.println(result);  
}
```

Lambda Expression

```
(num1, num2) -> {  
    int result = num1 + num2;  
    System.out.println(result);  
};
```

- Khi chỉ có một dòng biểu thức duy nhất bên trong hàm lambda, chúng ta có thể sử dụng cú pháp lambda biểu thức thay vì cú pháp câu lệnh lambda để đơn giản hơn.

```
public void add(int num1, int num2) {  
    int result = num1 + num2;  
    System.out.println(result);  
}
```

```
(num1, num2) -> System.out.println(num1+num2);
```

Biểu thức Lambda

Lợi ích

- Mã ngắn gọn: Giảm sự phức tạp khi viết mã.
- Dễ bảo trì: Code dễ đọc và dễ hiểu hơn.
- Tăng hiệu suất: Khi kết hợp với Stream API, biểu thức Lambda có thể tận dụng xử lý song song.

Hạn chế

- Khó debug: Với các biểu thức Lambda phức tạp, việc tìm lỗi có thể khó khăn.
- Khó đọc với người mới: Những người mới học Java có thể thấy cú pháp Lambda lạ lẫm.
- Hiệu suất: Trong một số trường hợp, sử dụng Lambda có thể làm tăng chi phí xử lý nếu không được tối ưu hóa.

Ứng dụng thực tế

Làm việc với Stream API

- Stream API trong Java 8 cho phép xử lý dữ liệu theo phong cách lập trình chức năng. Biểu thức Lambda thường được sử dụng để định nghĩa các thao tác như lọc, ánh xạ, hoặc sắp xếp.
- Ví dụ: Lọc các số chẵn trong danh sách:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);  
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

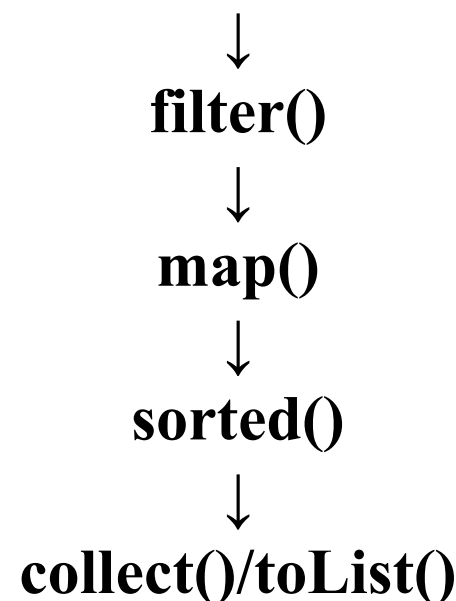
Stream là gì?

Hãy tưởng tượng Stream như một **dòng dữ liệu** chảy qua nhiều bước xử lý.

```
List<Integer> numbers = List.of(5, 4, 2, 1, 3);
```

```
List<Integer> result = numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .map(n -> n * 2)  
    .sorted()  
    .toList();
```

Danh sách



Các thao tác phổ biến

[5,4,2,1,3]

↓ filter

[4,2]

↓ map

[8,4]

↓ sorted

[4,8]

↓ toList

[4,8]

- filter()
- map()
- sorted()
- distinct()
- limit()
- skip()
- toList()
- collect()
- count()
- forEach()
- findFirst()
- reduce()

Sử dụng Stream

Xử lý Collection

- Lọc dữ liệu
- Chuyển đổi dữ liệu
- Thống kê, tổng hợp dữ liệu
- Tìm kiếm trong danh sách

Khi nào không nên lạm dụng?

- Thuật toán phức tạp
- Logic có nhiều trạng thái thay đổi
- Cần tối ưu hiệu năng ở mức rất thấp

Stream API

- Nguồn dữ liệu (Source)



- Xử lý trung gian (Intermediate Operations)



- Kết thúc (Terminal Operations)

Nguồn dữ liệu

- Tạo Stream

- Tạo Stream từ Collection

- `list.stream();`

- Ví dụ:

- ```
List<String> names = List.of("An", "Bình", "Cường");
names.stream();
```

- Tạo Stream trực tiếp : **Stream.of()**

- Ví dụ: `Stream.of("A", "B", "C");`

# Intermediate Operations - Các hàm biến đổi dữ liệu

Đặc điểm:

- Trả về Stream mới
- Chưa chạy ngay
- Chỉ chạy khi có Terminal Operation
- `filter()`, `map()`, `flatMap()`, `distinct()`, `sorted()`, ...

## Terminal Operations - Kích hoạt Stream chạy thật

- `forEach()`: Duyệt từng phần tử
- `collect()`: Thu kết quả về Collection
- `toSet()`, `toMap()`
- `count()`: Đếm số lượng
- `findFirst()`: Lấy phần tử đầu tiên.
- ...
- ...
- `reduce()`: Gộp nhiều phần tử thành một kết quả

# Function Interface

→ Trình biên dịch Java hoạt động mà không cần biết tên phương thức, kiểu dữ liệu tham số và kiểu trả về

- Một Function Interface là một giao diện chỉ định chính xác một phương thức trừu tượng duy nhất (Single Abstract Method - SAM).
- `@ FunctionalInterface` : Khai báo trên đầu Interface để đảm bảo rằng không thể thêm phương thức trừu tượng thứ hai.
- Chỉ cho phép 1 phương thức trừu tượng

# Function Interface

- $\text{Function}\langle T, R \rangle$  : Biến đổi dữ liệu
- $\text{Predicate}\langle T \rangle$  : Kiểm tra điều kiện
- $\text{Consumer}\langle T \rangle$  : Thực hiện hành động
- $\text{Supplier}\langle T \rangle$  : Sinh dữ liệu

## Function<T,R> : Biến đổi dữ liệu

- `Function<String, Integer> length = s -> s.length();`
- `System.out.println(length.apply("Java"));`
- Kết quả: 4

## Predicate<T> : Kiểm tra điều kiện

- Predicate<Integer> isAdult = age -> age >= 18;
- System.out.println(isAdult.test(20));
- Kết quả: True

## Consumer<T> : Thực hiện hành động

- `Consumer<String> printer = name -> System.out.println(name);`
- `printer.accept("Khuong");`
- Kết quả: print Khuong

## Supplier<T> : Sinh dữ liệu

- `Supplier<String> code = () -> "EMP001";`
- `System.out.println(code.get());`
- Kết quả: sinh code “EMP001”